VIGÍA: guía completa del proyecto

Nov 7, 2026 ·

1. Qué es VIGÍA

VIGÍA es un analista SOC N1 hecho con IA que trabaja las 24 horas para Tarvia: investiga cada alerta, cierra las falsas alarmas explicando por qué, descubre los ataques reales y deja al humano solo la decisión. Su lema: **la IA investiga, la persona decide**.

El problema de Tarvia (sale de `empresa_tarvia.md`):

- Unas 4.000 alertas al día antes de deduplicar y solo 3 analistas N1.
- El SOC funciona de 8:00 a 20:00 de lunes a viernes. De noche y en fin de semana nadie mira la consola, solo hay un N2 de guardia por teléfono.
- La CISO quiere cubrir las 24 horas y que el analista solo tenga que decidir.

El dato gancho, sacado de nuestro análisis: de las 138 alertas que forman los escenarios del reto, el **63%** salta de noche o en fin de semana. En el total de alertas es el 32%. Frase para la presentación: *Los atacantes de Tarvia trabajan cuando el SOC duerme.*

Dos modos de trabajo:

- **De día:** investiga cada alerta nada más saltar y se la pasa resuelta al analista, que revisa y decide en uno o dos minutos en vez de quince.
- **De noche y en fin de semana:** trabaja sola. Cierra el ruido, agrupa los ataques y, si algo es crítico, avisa al N2 de guardia como pide la política. Por la mañana deja la bandeja preparada.

Frase de cierre: *De día, VIGÍA hace más rápidos a vuestros analistas. De noche, hace que el SOC no se apague nunca.*

2. Qué nos dan: las 6 carpetas

El dataset cubre del 5 de octubre al 1 de noviembre de 2026, en la preparación del Black Friday. Hay que usarlo todo: cada fichero alimenta una pieza de VIGÍA.

Carpeta / fichero	Qué contiene	Pieza de VIGÍA que lo usa
`01_contexto/empresa_tarvia.md`	Negocio, qué proteger, horario del SOC y las **reglas internas** (adm.* solo desde JUMP01, AnyDesk no aprobado, etc.)	Motor de reglas, impacto en euros
`01_contexto/politica_soc.md`	Los 8 pasos del N1, matriz de severidad, SLA, quién aprueba cada contención, escalado obligatorio y agrupación	Motor de reglas, aprobación humana

Carpeta / fichero	Qué contiene	Pieza de VIGÍA que lo usa
`01_contexto/catalogo_casos_de_uso.csv`	Las 46 reglas del SIEM con táctica MITRE y runbook	Agente N1
`01_contexto/runbooks/RB-01..RB-10`	Qué comprobar y cómo decidir en cada tipo de alerta	Agente N1 (su guía)
`01_contexto/ticket_salida.schema.json`	Formato obligatorio del ticket N1	Agente N1, validador
`01_contexto/informe_incidente.schema.json`	Formato del informe N2 (fase 2)	Agente N2
`01_contexto/ticket_ejemplo.json`	Ticket modelo, incluido cómo justificar la severidad	Agente N1 (ejemplo)
`02_inventario/activos.csv`	1.185 equipos con criticidad, zona, servicio y propietario	Severidad, mapa
`02_inventario/usuarios.csv`	1.224 usuarios: rol, privilegios, VIP, MFA, notas	Contexto, reglas
`02_inventario/eventos_rrhh.csv`	Viajes, bajas, altas, teletrabajo	Benign true positive, insider
`02_inventario/calendario_cambios.csv`	Cambios y flujos aprobados (pentest, NESSUS, SCCM...)	Motor de reglas
`02_inventario/servicios_negocio.csv`	14 servicios con criticidad, impacto y responsable	Impacto, aprobaciones
`02_inventario/vulnerabilidades.csv`	CVE abiertas por equipo	Módulo CVE
`02_inventario/zonas_red.csv`, `tiendas.csv`	Las 10 zonas de red y las 28 tiendas	Mapa de Tarvia
`03_alertas/alertas.jsonl`	11.061 alertas (todas `new`)	Todo el sistema
`03_alertas/alertas_muestra_desarrollo.jsonl`	150 alertas para desarrollar, sin etiquetas	Pruebas y métricas
`04_logs/` (18 fuentes)	~800.000 eventos: EDR, proxy, DNS, firewall, correo, IdP, VPN, WAF, cloud, DLP, SQL, AI gateway, AD, CI/CD	Agente N1, cazador
`05_inteligencia/ti_feed.jsonl`	409 indicadores con reputación y confianza	Enriquecimiento

Carpeta / fichero	Qué contiene	Pieza de VIGÍA que lo usa
`05_inteligencia/geoip_asn.csv`	86.981 IPs: país, ASN, Tor, VPN, hosting	Enriquecimiento
`05_inteligencia/sandbox_informes.jsonl`	21 informes de ficheros	Enriquecimiento, preparación del directo
`06_herramientas/mock_soc_api.py`	API simulada del SOC con alertas, CMDB, logs, intel, cambios, aprobaciones y contención	Herramientas del agente

Ojo con la API simulada: la búsqueda en logs recorre los eventos uno a uno y será lenta con 800.000. La ampliamos con consultas sobre DuckDB. Las contenciones requieren `approval_token` salvo `block_ioc` y `purge_email`, y la API rechaza actuar sobre la red OT.

3. Qué hay escondido en los datos

De 11.061 alertas, la gran mayoría es ruido explicable y unas 138 forman 12 incidentes reales y varias trampas. Las fechas están en UTC; la hora local es UTC+2 hasta el 25/10 y UTC+1 después.

3.1 El ruido

Regla	Alertas	Por qué es ruido (a confirmar con los logs)
UC-WEB-001	3.406	Escaneos de Internet bloqueados por el WAF (salvo las del pentest, que son BTP)
UC-EML-001	2.295	Usuarios reportando newsletters como phishing
UC-EDR-002	1.349	PowerShell codificado de SCCM, flujo aprobado CHG-2026-1031
UC-IDP-001	901	Viaje imposible: hay que cruzar con viajes de RRHH y VPN
UC-NET-001	746	Beaconing a Microsoft, Teams, Google, BMC e impresoras (salvo 2 reales)
UC-NET-005	561	Dominios con aspecto DGA de servicios conocidos
UC-EDR-001	532	Excel lanzando saplogon.exe, firmado por SAP (salvo 1 real)
UC-AI-001	415	Clientes que se corrigen ("olvida la talla que te dije"); solo 2 son inyecciones
UC-EDR-003	411	PUA bloqueadas o en cuarentena (eicar, KMSPico, crack...)
UC-NET-006	91	DNSBL y antivirus (salvo 12 de WMS01, reales)
UC-CLD-003	50	Política de bucket hecha por terraform desde el runner CI

3.2 Los 12 incidentes reales

#	Incidente	Fechas	Cadena	Impacto
1	AiTM + fraude de IBAN	7-8/10	Sesión robada de laura.ferrer2 desde hosting con python-requests > regla de buzón que oculta "Montroig/IBAN" > petición de cambio de IBAN por 148.320 € a Lituania (jordi.serra2)	Fraude económico, critical
2	Latrodectus > preransomware	9-18/10	Adjunto Pedido_Q4_2026.js a marta.puig (WS-BCN-1770) > C2 aplihartom.com > Cobalt Strike > reconocimiento el sábado a las 04:13 > LSASS con credenciales de adm.aortega > PsExec a FS01 > borrado de shadow copies (bloqueado) > spoolhlp.exe en PRINT01 > 18/10 svc_print_upd en Domain Admins y EDR detenido	Dominio comprometido, critical
3	Minero en Jenkins	6/10	JENKINS-DEV01 expuesto con CVE-2024-23897 > java > kswapd0 (XMRig) con crontab	Recursos, high
4	Cadena de suministro > AWS > CRM	13-16/10	Paquete tarvia-checkout-utils del registro público (dependency confusion) lee claves AWS > uso desde Kali > usuario backup-sync con AdministratorAccess > ~112 GB de copias del CRM	Datos de 1,3 M clientes, AEPD 72 h
5	Credential stuffing + ATO	10-11/10	3.200 fallos/15 min desde proxies residenciales, 26 accesos correctos > 12 cuentas con cambio de email y compra de tarjetas regalo	Fraude a clientes
6	Proveedor Logisoft comprometido	11-29/10	VPN de madrugada desde 77.90.185.20 > servicio wmsupd.exe sin firma en WMS01 (**sin alerta propia**) > sqlcmd sobre direcciones de clientes > túnel DNS cdn-wms-metrics cada noche	Datos de clientes, junto a OT
7	Tarjetas robadas en la tienda 7	12-21/10	AnyDesk con control desde Marruecos en LT-T07-02 > possvc.exe (RAM scraper) en 3 TPV que dice ser "instalado por Seguridad" > beacon pos-update-service	PCI DSS, critical

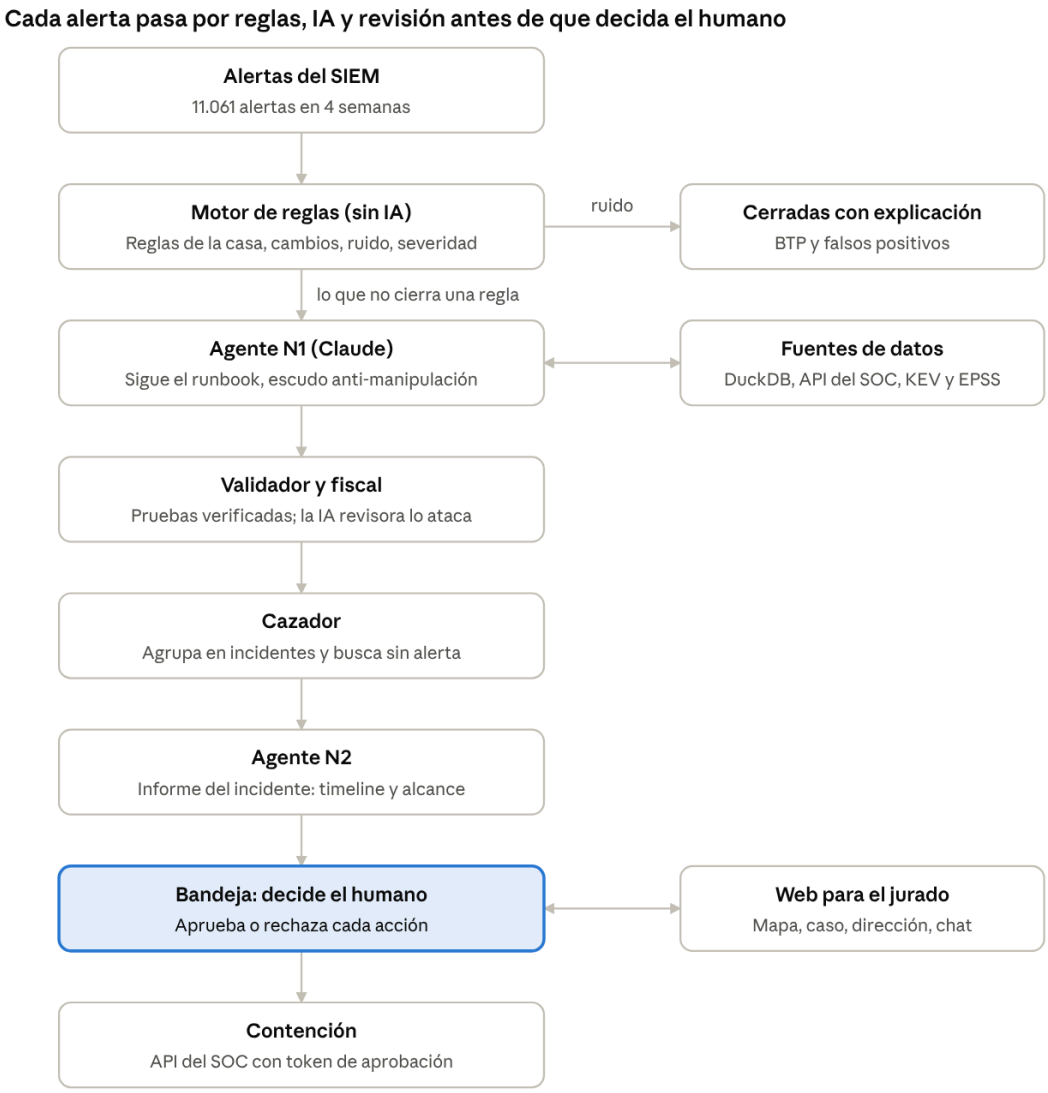
#	Incidente	Fechas	Cadena	Impacto
8	Insider que se va	14-15/10	núria.vidal (último día 16/10) sube backup_clientes_oct.7z cifrado a mega.nz y envía 41.200 emails de clientes a su correo personal	RGPD, DPO
9	svc_backup desde Tor	13/10	Cuenta sin MFA (riesgo aceptado) entra por VPN desde un nodo Tor de madrugada	Cuenta privilegiada
10	Fatiga MFA	10/10	27 push en 9 min a sergio.navarro un sábado, aprobado al final, desde VPN comercial	Cuenta comprometida
11	Inyección y fuga en Vela	8/10	Inyección bloqueada, intento de leer pedidos de otro cliente, y el asistente revela códigos de descuento internos	Fuga de info interna
12	Campaña de phishing packaging-iberia	20/10	45 correos con texto oculto dirigido al clasificador automático de seguridad; sesión AiTM de lucia.castro	Intento de engañar a la IA

3.3 Las trampas (benign true positive o falso positivo)

- Pentest de la tienda online del 8 al 10/10 desde 192.0.2.10-12 (CHG-2026-1042).
- NESSUS01 escaneando los miércoles de 01:00 a 05:00.
- PsExec desde JUMP01 con adm.dnavarro el 13/10 (CHG-2026-1057).
- Mantenimiento OT desde JUMP01 el 17/10 (CHG-2026-1063) y ENG-WS01 hablando con OT (autorizado).
- Alta en Backup Operators el 23/10 (CHG-2026-1078) y paradas del EDR el 24/10 (CHG-2026-1080).
- Logisoft fuera de horario el 16/10 desde su IP corporativa (CHG-2026-1071); los demás accesos de 18:0x desde su IP son leves.
- Nóminas a gestoria-pons (FLUJO-0007).
- puppeteer, sharp y esbuild en CI: paquetes conocidos que descargan binarios por diseño.
- Viaje de ricard.castells a Nueva York del 11 al 14/10.
- Nodos Tor, proxies residenciales y VPN comercial: sospechosos, no maliciosos por sí mismos. mega.nz, WeTransfer y AnyDesk: legítimos, depende del uso.

4. Arquitectura: cómo fluye una alerta

Una alerta recorre una cadena: primero reglas fijas, luego la IA, luego una revisión, y al final decide el humano.



arquitectura de VIGÍA · 8 pasos en cadena

El motor de reglas cierra el ruido y lo autorizado sin gastar IA. Lo que no puede cerrar con una regla pasa al agente N1, que consulta los datos con herramientas. El validador y el fiscal comprueban cada veredicto, el cazador une alertas en incidentes y el agente N2 escribe el informe. Nada disruptivo se ejecuta sin la aprobación del humano en la bandeja.

5. Pieza a pieza

Cada pieza tiene una entrada, una salida y un responsable. Lo que se puede decidir con una regla fija va en código; la IA solo decide lo que necesita criterio.

5.1 Ingesta de datos (DuckDB)

- **Qué hace:** carga las alertas, el inventario, la inteligencia y los 18 logs en una base DuckDB, con índices por `event_id`, host, usuario, IP, dominio, hash y fecha.
- **Por qué:** la API simulada busca texto evento a evento. Con DuckDB, "todo lo de WS-BCN-1770 en ±24 h" tarda milisegundos.
- **Salida:** `tarvia.duckdb` + funciones de consulta que usarán el agente, el cazador y la web.
- **Normalización:** todas las fechas a UTC y una columna de hora local de Madrid (UTC+2 hasta el 25/10, UTC+1 después) para saber si fue fuera de horario.

5.2 Motor de reglas (sin IA)

Tres partes, todas en Python y con su explicación por escrito en cada decisión:

1. **Reglas de la casa y cambios aprobados.** Una lista de comprobaciones sacadas de `empresa_tarvia.md`, `calendario_cambios.csv`, `usuarios.csv` y `eventos_rrhh.csv`. Ejemplos: IP origen dentro del pentest y en su ventana → BTP; NESSUS01 en miércoles 01-05 → BTP; PowerShell codificado con padre CcmExec.exe → BTP (CHG-2026-1031); cuenta adm.* desde un equipo distinto de JUMP01 → sospechoso; AnyDesk → nunca aprobado; ext.logisoft fuera de L-V 8-18 sin cambio → sospechoso; acceso a OT desde algo que no sea ENG-WS01 o WMS01 sin cambio → escalar.
2. **Filtro de ruido.** Patrones conocidos y explicables: WAF con `action: blocked` en rutas que no existen; EDR con `quarantined`/`blocked` y sin actividad posterior; Excel > saplogon firmado por SAP; beaconing a dominios corporativos conocidos. Cada patrón cuenta cuántas alertas cierra.
3. **Calculadora de severidad.** Aplica la matriz de `politica_soc.md`: severidad de la regla × criticidad del activo o identidad más crítica, con un ajuste máximo de ±1 (+1 si hay ejecución o acceso confirmado, identidad VIP o privilegiada, o datos personales o pagos; −1 si un control lo bloqueó sin efecto). Escribe el `severity_rationale` igual que el ticket de ejemplo: "Regla low sobre activo crítico = medium; −1 por bloqueo del WAF".

Regla de oro: el motor nunca cierra un true positive. Si una alerta no encaja limpiamente en una regla, pasa a la IA.

5.3 Agente N1 (la IA investigadora)

- **Entrada:** una alerta (o un grupo) que el motor no ha podido cerrar.
- **Cómo trabaja:** Claude con herramientas (tool use) que llaman a la API del SOC y a DuckDB: `get_alert`, `get_asset`, `get_user`, `search_logs`, `get_intel`, `get_sandbox`, `get_changes`, `get_cve`. En el prompt de sistema lleva la política y el runbook de esa regla (RB-01 a RB-10) y sigue los 7 pasos: entender, contextualizar, enriquecer, correlacionar ±24 h, decidir, severidad y documentar.
- **Salida:** un ticket que cumple `ticket_salida.schema.json`: título que dice qué pasa (no qué regla saltó), veredicto, confianza, severidad calculada por el motor, resumen de 3-5 frases, `evidence` con `event_id`, `enrichment`, MITRE, acciones recomendadas con `requires_approval`, escalado, preguntas abiertas y `triage_seconds`.

- **Validador de evidencias:** antes de aceptar el ticket, el código comprueba que cada `event_id` citado existe en los logs y dice lo que el agente afirma. Si falla, el ticket vuelve al agente o pasa a revisión humana. Así la IA no puede inventarse nada.
- **Validador de schema:** el JSON se valida contra el schema; si no cumple, se repite.

5.4 Módulo CVE

- **Qué hace:** cuando la alerta toca un equipo con CVE abiertas en `vulnerabilidades.csv`, consulta **CISA KEV** (¿se está explotando de verdad?) y **EPSS** (probabilidad de explotación en 30 días). Ambos son gratis y sin clave.
- **Efecto:** sube la prioridad y lo explica. Ejemplo: JENKINS-DEV01 tiene CVE-2024-23897 abierta desde febrero, expuesto a Internet: puerta abierta.
- **Precauciones:** copia local de KEV y EPSS por si falla Internet. Algunas CVE del dataset parecen inventadas (CVE-2026-20817, CVE-2026-21311): si no hay datos externos, se dice "sin datos externos".

5.5 Fiscal (la IA revisora)

- **Qué hace:** otra llamada a Claude, con instrucciones de intentar **refutar** el veredicto con los mismos datos: "¿hay un cambio aprobado que lo explique?", "¿la prueba citada dice realmente eso?".
- **Resultado:** si no consigue tumbarlo, la confianza sube. Si lo consigue, el ticket va a revisión humana con los dos argumentos lado a lado.
- **Cuándo:** solo en tickets de severidad medium o superior, para ahorrar coste.

5.6 Escudo anti-manipulación

- **Problema:** el atacante escribe asuntos, cuerpos de correo, prompts a Vela y descripciones de servicios. Hemos visto 45 correos con texto oculto dirigido a la IA y un malware que se describe como "instalado por Seguridad, no requiere revisión".
- **Qué hacemos:** los campos controlados por el atacante se pasan a la IA marcados como **datos no confiables**, nunca como instrucciones. Un detector (patrones + IA) busca texto que intente dar órdenes al clasificador, HTML oculto o afirmaciones de autoridad sin respaldo (un ticket SEC-2210 que no existe en el calendario).
- **Giro:** si se detecta, se añade como **evidencia contra el atacante** y sube la confianza del true positive. Es uno de nuestros momentos fuertes de la demo.
- **Vela:** distinguir al cliente que se corrige ("olvida la talla") del ataque real leyendo la sesión entera, como pide RB-07.

5.7 Cazador y correlación

- **Agrupación:** construye un grafo de entidades (usuario, equipo, IP, dominio, hash, cuenta cloud). Dos alertas que comparten entidad en una ventana de tiempo van al mismo ticket, como pide la política ("diez tickets de un incidente son diez veces más trabajo").
- **Cadenas largas:** sigue las entidades a lo largo de semanas. Ejemplo: el hash de svchelper.exe del 10/10 reaparece como spoolhlp.exe en PRINT01 y lleva a la cuenta creada el 18/10.

- **Caza sin alerta:** consultas fijas que buscan lo que no saltó: servicios instalados sin firma, cuentas nuevas sin propietario (svc_print_upd), accesos de proveedores desde ASN nuevos, procesos que leen bases de datos de clientes. Así se encuentra wmsupd.exe en WMS01.

5.8 Agente N2 (informe de incidente)

- **Entrada:** un grupo de tickets que forman un incidente.
- **Salida:** un informe que cumple `informe_incidente.schema.json`: paciente cero, acceso inicial, línea de tiempo con `event_id` y MITRE, alcance (usuarios, equipos, datos, impacto de negocio), causa raíz, plan de contención con aprobador e impacto, erradicación, notificaciones (DPO y AEPD/CNPD en 72 h, adquirente PCI, proveedor), huecos de detección y preguntas abiertas.
- **Mejora de reglas:** propone qué ajustar en el SIEM. Ejemplo: "UC-EDR-002 generó 1.349 falsos positivos por SCCM: excluir padre CcmExec.exe con usuario SYSTEM".

5.9 Aprobación humana y contención

- La tabla de aprobaciones de la política va en código: `block_ioc` y `purge_email` se ejecutan solos; aislar, deshabilitar, revocar sesiones o desactivar claves se proponen y esperan.
- En la web, el analista ve la acción, el objetivo, la justificación y el **radio de impacto** ("aislar PAY-GW01 para los cobros online"), y pulsa aprobar. Eso llama a `POST /approvals`, que devuelve el token, y después a `POST /containment/{accion}`.
- Nunca se actúa sobre OT: solo se escala a Operaciones.
- Nunca bloquear globalmente servicios de uso dual (mega.nz, AnyDesk) sin N2.

5.10 Métricas honestas

- Etiquetamos a mano las 150 alertas de la muestra de desarrollo y los 12 incidentes.
- Medimos precisión y recall del veredicto, acierto de severidad, agrupación correcta y tiempo de triaje por alerta.
- Una diapositiva con lo que VIGÍA **no** resuelve y por qué.

6. La web: lo que verá el jurado

Cinco pantallas en una sola web, ligera como la del ganador del año pasado (HTML, CSS y JS contra el backend FastAPI). La visualización es obligatoria en el reto y pesa en la nota.

1. **Mapa de Tarvia.** Las 10 zonas de `zonas_red.csv` dibujadas como un mapa (oficinas de Barcelona y Madrid, CPD, DMZ, tiendas, almacén IT, OT, desarrollo, VPN y AWS). Un deslizador recorre las 4 semanas hora a hora. Las alertas caen como puntos: grises las cerradas, rojas las escaladas. Los incidentes se dibujan como flechas que saltan entre zonas (por ejemplo, WS-BCN-1770 → FS01 → PRINT01). Los equipos con CVE explotadas brillan como "puertas abiertas". Una franja oscura marca las horas sin SOC para que se vea que los ataques caen ahí.
2. **El caso.** Al pulsar un incidente: línea de tiempo con MITRE, grafo de entidades (usuario, equipo, IP, dominio) y el razonamiento de VIGÍA paso a paso ("consulté la CMDB → FS01 es crítico", "miré el calendario → no hay cambio"). Cada prueba es clicable y abre el evento original. Al lado, lo que dijo el fiscal. Si hubo intento de manipulación, aparece un aviso destacado.

3. **Bandeja de la mañana.** Lista de tickets ordenada por severidad y SLA, con veredicto, confianza y acciones propuestas. Botón de aprobar o rechazar cada acción, con el radio de impacto delante. Muestra el registro de contenciones ejecutadas.
4. **Panel para la dirección.** Números grandes: alertas recibidas frente a casos para el humano, horas sin vigilancia cubiertas, euros en riesgo (148.320 € del IBAN, 120.000 €/h de tienda caída), horas de analista ahorradas y el **reloj RGPD**: cuenta atrás de 72 h desde que se detecta una fuga de datos personales, con a quién notificar.
5. **Chat "Pregúntale a VIGÍA".** El analista pregunta en lenguaje natural ("¿qué hizo marta.puig la semana pasada?") y VIGÍA responde con datos consultados con las mismas herramientas, citando eventos.

Modo replay para la demo: un botón reproduce un fin de semana acelerado. La cola se llena, VIGÍA la va vaciando y el contador de tiempo ahorrado sube.

7. Stack y estructura del repositorio

Stack simple y probado, igual de ligero que el del ganador: nada que pueda romperse en directo.

Capa	Tecnología	Para qué
Lenguaje	Python 3.11+	Todo el backend
Datos	DuckDB	Consultas rápidas sobre los 800.000 eventos
IA	API de Claude (Anthropic) con tool use	Agente N1, fiscal, agente N2, chat
Backend	FastAPI + Uvicorn	Amplía la API simulada y sirve a la web
Validación	jsonschema + Pydantic	Tickets e informes con el schema exacto
Frontend	HTML, CSS y JS + Cytoscape.js (grafo) + ECharts (líneas de tiempo y gráficos)	Las 5 pantallas
Inteligencia externa	CISA KEV y EPSS, con copia local	Módulo CVE
Equipo	Git y GitHub, ramas por pieza	Trabajar los 5 a la vez

```
vigia/
├── apps/
│   ├── backend/
│   │   ├── main.py              # FastAPI: endpoints para la web
│   │   ├── ingest/              # carga a DuckDB
│   │   ├── rules/               # reglas de la casa, ruido, severidad, aprobaciones
│   │   ├── agents/              # N1, fiscal, N2, chat + prompts por runbook
│   │   ├── tools/               # herramientas del agente (API + DuckDB + CVE)
│   │   ├── shield/              # escudo anti-manipulación
│   │   ├── hunt/                # grafo de entidades, agrupación, caza
│   │   └── validate/            # schema y validador de evidencias
```

```
│   └── frontend/
│       ├── index.html, app.js, styles.css
│       └── views/               # mapa, caso, bandeja, dirección, chat
├── data/                        # dataset de Deloitte (NO se sube a Git)
├── out/                         # tickets e informes generados
├── eval/                        # etiquetas a mano y métricas
├── scripts/                     # run_all, replay, preparar alertas del directo
├── docs/                        # arquitectura y decisiones
└── README.md
```

Reglas del repo: la clave de la API en una variable de entorno, nunca en el código. El dataset fuera de Git. Scripts de arranque para Windows y Linux, como hizo el ganador.

8. Reparto en 5 personas y plan de 32 horas

Cada persona es dueña de una pieza de principio a fin, y todas trabajan a la vez desde la hora 1. El nivel 1 tiene que funcionar perfecto antes de tocar lo vistoso.

Persona	Pieza	Entrega	Lo que presenta
P1 Datos y reglas	5.1, 5.2, 5.9	DuckDB, motor de reglas, severidad, aprobaciones	El filtro: el pentest y los escaneos que no eran ataques
P2 Agentes	5.3, 5.4, 5.5	Agente N1, validadores, módulo CVE, fiscal	El robo de 148.320 € investigado paso a paso
P3 Caza y N2	5.6, 5.7, 5.8	Escudo, grafo de entidades, caza, informes N2	El proveedor del almacén que no tenía alerta y los correos que intentan engañar a la IA
P4 Frontend	6	Las 5 pantallas y el modo replay	El Mapa de Tarvia en movimiento
P5 Demo y calidad	5.10 + presentación	Etiquetas, métricas, alertas sintéticas del directo, diapositivas, vídeo de respaldo	El robo del CRM, el reloj RGPD y las métricas honestas

Contratos entre personas (decidir en la hora 1): el formato del ticket (el schema), las funciones de consulta de DuckDB y los endpoints que usará la web. Con eso, P4 puede trabajar con datos falsos desde el principio.

Plan por bloques:

1. **Horas 0-2. Arranque.** Repo, entorno, clave de la API, contratos. P1 carga DuckDB. P5 empieza a etiquetar la muestra de desarrollo. P4 maqueta las pantallas con datos falsos.
2. **Horas 2-8. Nivel 1 base.** Motor de reglas y severidad (P1). Agente N1 con 3 runbooks (phishing, EDR, identidad) y validadores (P2). Grafo de entidades (P3). Bandeja y caso (P4).

3. **Control hora 8:** una alerta entra y sale como ticket válido con pruebas verificadas. Si no, todo el equipo ayuda hasta que salga.
4. **Horas 8-16. Nivel 1 completo.** Los 10 runbooks, el filtro de ruido entero, fiscal (P2). Agrupación y escudo (P3). Mapa de Tarvia (P4). Primeras métricas (P5).
5. **Control hora 16:** las 11.061 alertas procesadas de punta a punta y métricas sobre la muestra etiquetada.
6. **Horas 16-24. Diferenciadores.** Caza sin alerta e informes N2 (P3). Módulo CVE (P2). Aprobación y contención (P1). Panel de dirección, reloj RGPD y replay (P4). Alertas sintéticas del directo (P5).
7. **Horas 24-28. Congelar.** No se añade nada nuevo. Solo arreglar fallos, probar las alertas del directo y grabar un vídeo de respaldo de la demo.
8. **Horas 28-32. Presentación.** Diapositivas, guion por persona y dos ensayos completos cronometrados.

Turnos de sueño: de dos en dos, nunca a la vez los dueños de piezas que dependen entre sí (P1 y P2).

9. Diferenciación, demo, presentación y riesgos

9.1 Por qué no hacemos lo mismo que los demás

Lo que harán la mayoría	Lo que hace VIGÍA
La IA clasifica alertas una a una	Caza: une alertas en ataques completos y encuentra los que no tienen alerta
"Es malicioso" y hay que creerla	Se ve pensar: cada afirmación con su prueba verificada, y un fiscal que intenta tumbarla
La IA lee lo que escribe el atacante	No se deja engañar y usa el intento como prueba
Habla de alertas y severidades	Habla de euros, plazos legales y a quién avisar
Se queda en el ticket N1	Llega al informe N2 (fase 2)
Solo mira dentro	Cruza las vulnerabilidades con las que se explotan hoy en el mundo
Sigue las reglas	Propone cómo mejorarlas
Solo enseña lo que funciona	Enseña métricas reales y lo que no resuelve

9.2 Preparación de las 10 alertas en directo

La sandbox tiene informes de 3 ficheros que no aparecen en ningún log: un instalador falso de Notepad++ (Lumma Stealer), la web shell `cache_helper.php` y el PDF `Actualizacion_nomina_octubre.pdf` con un QR. Además, WEB-ECOM03 tiene un plugin vulnerable desde el 15/10. Probablemente salgan en el directo. P5 crea alertas sintéticas de esos casos y las pasamos por VIGÍA muchas veces antes.

9.3 La presentación: "Una noche en Tarvia"

1. **Apertura (P5):** el problema y el dato del 63%. Se apagan las luces del SOC en el mapa.
2. **El filtro (P1):** 11.061 alertas se convierten en unas pocas; el pentest y NESSUS se cierran solos con su explicación.
3. **La investigación (P2):** el fraude de 148.320 € paso a paso, con el fiscal intentando tumbarlo.
4. **La caza (P3):** el proveedor del almacén sin alerta y los correos que intentan engañar a la IA.
5. **El mapa (P4):** el ransomware moviéndose durante 9 días por la empresa, de madrugada.
6. **Cierre (P5):** el robo del CRM, el reloj RGPD, el humano pulsando aprobar, métricas honestas y la frase final.
7. **Las 10 alertas en directo** por el mismo flujo.

9.4 Riesgos y plan B

Riesgo	Plan B
Falla Internet o la API de Claude en la demo	Resultados ya generados en `out/` y vídeo grabado
La IA se equivoca con una alerta del directo	El fiscal la marca para revisión humana: lo contamos como parte del diseño
No da tiempo a todo	Orden de corte: replay, chat, CVE. El nivel 1 y el mapa no se tocan
Coste o límites de la API	El motor de reglas filtra antes; el fiscal solo en medium o superior
Los IOCs reales del dataset	Solo consultas de reputación, nunca visitarlos ni descargar nada

Regla final: un nivel 1 sólido vale más que un nivel 4 a medias. Lo dice el propio reto.